

Po zestawieniu **poprawionego GMRES(20)+element LU**, **poprawionego BiCGSTAB+element LU** i tego, co masz obecnie w Vanadisie, wychodzi dość wyraźny obraz:

**Do obecnego dużego Vanadisa/OpenMP/CUDA zostawiłbym aktualny DBCG + Jacobi.
Z dwóch historycznych alternatyw BiCGSTAB jest znacznie ciekawszy praktycznie niż GMRES(20).**

Sam elementowy LU jest numerycznie mocniejszy od Jacobi, ale architektonicznie niezbyt pasuje do obecnego Vanadisa na dużych siatkach/GPU.

Porównanie

Cecha	Obecny Vanadis: DBCG + Jacobi	Poprawiony BiCGSTAB + LU	Poprawiony GMRES(20) + LU
Metoda	BiCG	BiCGSTAB	Restarted GMRES(20)
Macierz globalna	nie	nie	nie
EBE	tak	tak	tak
A x / iterację	1	2	1
A ^T x / iterację	1	0	0
Preconditioner	Jacobi	elementowe LU (8x8)	elementowe LU (8x8)
Siła preconditionera	słaba	silna lokalnie	silna lokalnie
Koszt preconditionera	minimalny	duży	duży
Krylov vectors	6	7	~44
Pamięć	najmniejsza	średnia + duże LU	bardzo duża
Orthogonalizacja	nie	nie	tak
Globalne redukcje	mało	umiarkowanie	dużo
Breakdown	możliwy	możliwy, ale obsługowany	mniej typowy
Odporność dla trudnych A	średnia	dobra	najlepsza
CUDA	bardzo dobra, już działa	potencjalnie dobra	najśłabsza z trzech
Stan w Vanadis	produkcyjny	eksperymentalny	eksperymentalny

1. Co właściwie masz teraz w Vanadisie

Aktualna metoda to nie CG dla macierzy symetrycznej, tylko BiConjugate Gradient dla macierzy niesymetrycznej, co ma sens dla układu adwekcja-dyfuzja-DR.

W każdej iteracji mniej więcej wykonywane są operacje:

$$\mathbf{z} = \mathbf{M}^{-1} \mathbf{r}$$

$$\mathbf{p} = \mathbf{z} + \beta \mathbf{p}$$

$$\mathbf{q} = \mathbf{A} \mathbf{p}$$

oraz dla układu dualnego:

$$\mathbf{z}^{\sim} = \mathbf{M}^{-T} \mathbf{r}^{\sim}$$

$$\mathbf{p}^{\sim} = \mathbf{z}^{\sim} + \beta \mathbf{p}^{\sim}$$

$$\mathbf{q}^{\sim} = \mathbf{A}^T \mathbf{p}^{\sim}$$

Preconditioner jest bardzo prosty: $\mathbf{M} = \mathbf{D} = \text{diag}(\mathbf{A})$, więc $z_i = r_i / D_i$. Przekątna nie jest przypadkową aproksymacją - jest składana elementowo i odpowiada dokładnej globalnej przekątnej operatora EBE.

$$\begin{aligned} z(i) &= r(i) / \text{diag}(i) \\ zz(i) &= rr(i) / \text{diag}(i) \end{aligned}$$

To jest bardzo tani preconditioner.

2. DBCG ma jedną wadę, której tamte dwie metody nie mają

Obecny DBCG potrzebuje zarówno:

$$A x$$

jak i:

$$A^T x$$

Dlatego masz dwa operatory: `matvec_0` i `mttvec_0`. W EBE nie jest to jednak tak bolesne jak przy CSR, bo nie trzeba przechowywać osobnej macierzy transponowanej. Transponowane działanie operatora powstaje z lokalnych bloków 8x8.

Obecna implementacja Dirichleta jest pod tym względem spójna: $A_D = P A P$, a transpose $A_D^T = P A^T P$. Potrzeba A^T jest więc wadą wydajnościową, ale nie stanowi poważnego problemu architektonicznego w EBE.

3. BiCGSTAB + element LU

To jest najciekawsza z dwóch historycznych alternatyw.

$$p_hat = M^{-1} p$$

$$v = A p_hat$$

$$s = r - \alpha v$$

$$s_hat = M^{-1} s$$

$$t = A s_hat$$

Czyli jedna iteracja kosztuje:

$$2 \times A + 2 \times M^{-1}$$

ale nie potrzebuje A^T . To jest atrakcyjne dla GPU.

Koszt pamięci elementowego LU

Dla każdego elementu przechowywany jest blok LU 8x8 oraz 8 pivotów. W poprawionej wersji jest to 64 liczb double na element plus 8 liczb całkowitych.

$$64 \times 8 + 8 \times 4 = 544 \text{ B / element}$$

Dla 1 miliona elementów daje to około 544 MB tylko na preconditioner. Dla 2 milionów elementów daje to około 1.09 GB.

Samo `bb` już przechowuje 64 wartości double na element:

$$64 \times 8 = 512 \text{ B / element}$$

<code>bb</code>	512 MB / mln elementów
<code>element LU</code>	544 MB / mln elementów

<code>razem</code>	1056 MB / mln elementów

To robi się bardzo nieprzyjemne przy ograniczonej pamięci GPU.

4. Jacobi na tym tle prawie nic nie kosztuje

Dla Jacobi przechowujesz tylko `diag(N)`, czyli 8N bajtów. Dla $N = 2.8$ mln węzłów to około 22.4 MB.

Jacobi ma ogromną przewagę pamięciową

Nawet jeśli elementowy LU jest znacznie mocniejszym preconditionerem numerycznym, różnica pamięciowa jest ogromna.

5. A dodatkowo masz teraz Picarda

W v2026.3.0 przy nieliniowym $P = P(S)$ macierz zmienia się między iteracjami Picarda. To jeszcze bardziej przemawia przeciw elementowemu LU.

Przy każdym rozwiązaniu liniowym należałoby ponownie wykonać faktoryzację $K_e \rightarrow LU_e$ dla wszystkich elementów. W aktualnym teście nieliniowym wykonano 71 rozwiązań liniowych w 20 krokach czasu.

Jacobi powstaje praktycznie za darmo już podczas montażu przekątnej, a jego zastosowanie to tylko jedno dzielenie na stopień swobody.

6. GMRES(20) + LU

Numerycznie jest to prawdopodobnie najmocniejsza z trzech metod. GMRES w każdym kroku minimalizuje residual w aktualnej przestrzeni Kryłowa i zwykle dobrze zachowuje się dla trudnych, niesymetrycznych i niedominujących diagonalnie macierzy.

Cena jest jednak wysoka. W poprawionej wersji przechowywane są m.in. $v(N,21)$, $z(N,20)$, w , r i dodatkowe wektory robocze.

około 44 N liczb double = 352 N bajtów

Dla miliona węzłów to około 352 MB samych dużych wektorów GMRES. Do tego dochodzi elementowe LU. Jest to zupełnie inna klasa pamięciowa niż obecny DBCG + Jacobi.

7. GMRES ma problem szczególnie nieprzyjemny dla GPU

W każdej iteracji wykonuje ortogonalizację Arnoldiego:

$$h_{ij} = v_i^T w_j$$

Dla iteracji j trzeba przejść przez $v_1 \dots v_j$. Przy reortogonalizacji może to zostać wykonane drugi raz.

Oznacza to wiele globalnych iloczynów skalarnych, redukcji, odczytów pamięci i synchronizacji GPU. Dla GMRES(20) pod koniec cyklu może pojawić się do 20 dużych operacji ortogonalizacyjnych.

GMRES(20) jest z tych trzech metod najmniej atrakcyjny dla GPU

8. BiCGSTAB pod względem pamięci jest znacznie lepszy

Poprawiona wersja przechowuje siedem dużych wektorów: r , r_0 , p , v , $phat$, $shat$ i tt .

$$7 N \times 8 = 56 N B$$

Obecny DBCG przechowuje sześć dużych wektorów p , pp , r , rr , z , zz oraz $diag(N)$. Łącznie daje to również około siedmiu wektorów typu double.

Czyli same wektory Kryłowa BiCGSTAB i obecnego DBCG kosztują prawie tyle samo. Problemem BiCGSTAB + LU nie są wektory, tylko pamięć LU_e .

9. Numeryczna siła preconditionera

element LU >> Jacobi

Jacobi widzi tylko A_{ii} . Element LU widzi pełną lokalną macierz $K_e(8 \times 8)$, a więc lokalne sprzężenia pomiędzy ośmioma węzłami HEX8. Dla adwekcji i stabilizacji DR może to bardzo wyraźnie zmniejszyć liczbę iteracji.

Nie da się jednak z samego kodu uczciwie stwierdzić, o ile liczba iteracji spadnie. To trzeba zmierzyć na dokładnie tej samej macierzy i RHS.

10. Obecny DBCG już zbiega całkiem dobrze

W aktualnym teście nieliniowym wykonano 71 rozwiązań liniowych i 1774 iteracje DBCG. Wszystkie rozwiązania osiągnęły tolerancję $1e-8$.

1774 / 71 $\approx$ 25.0 iteracji na jedno rozwiązanie liniowe

Okolo 25 iteracji nie sugeruje, że Jacobi jest dramatycznie za słaby. Gdyby solver wymagał kilkuset iteracji, elementowy LU byłby znacznie bardziej kuszący.

11. Historyczne GMRES i BiCGSTAB nie są drop-in replacement dla obecnego v2026.3.0

Historyczne procedury używały matvec, natomiast obecny Vanadis narzuca jednorodny Dirichlet przez matvec_0, mttvec_0 i bc_dirichlet, a przed solverem ustawia na węzłach Dirichleta RHS=0, diag=1 i x=0.

Jeżeli GMRES albo BiCGSTAB miałyby być naprawdę wstawione do obecnego Vanadisa, należałoby przenieść również projekcję P A P do ich operatora oraz właściwie zbudować preconditioner na ograniczonej przestrzeni swobodnych stopni swobody.

Nie należy wkładać tych historycznych procedur 1:1 do v2026.3.0.

12. Co jest najlepsze w praktyce

1) DBCG + Jacobi - najlepszy jako obecny solver produkcyjny

- Masz już wersję CPU i CUDA.
- Zachowuje pełną architekturę EBE.
- Działa z wariantem atomic i graph coloring.
- Ma bardzo mały narzut pamięciowy.
- Jest sprawdzony w aktualnym kodzie i testach.
- W aktualnym teście potrzebuje średnio około 25 iteracji na solve.

Zmiana solvera nie ma obecnie uzasadnienia funkcjonalnego.

2) BiCGSTAB - najciekawsza alternatywa eksperymentalna

Najciekawszy eksperyment dla obecnego Vanadisa to nie BiCGSTAB + element LU, lecz:

BiCGSTAB + obecny Jacobi

Dałoby to brak A^T , brak mttvec, tę samą diag, prawie tę samą pamięć, prostą ścieżkę CUDA i dwa matvec na iterację - bez potężnego kosztu pamięciowego elementowego LU.

To byłoby najbardziej uczciwe porównanie algorytmu z aktualnym DBCG, ponieważ zmieniłby się solver, ale nie preconditioner i model pamięci.

3) GMRES(20) + LU - dobry solver badawczy/CPU

Nadaje się do sprawdzania trudnych macierzy, benchmarku innych solverów, sytuacji, w których BiCG/DBCG nie zbiega, oraz małych i średnich problemów CPU.

Nie wybrałbym go jako głównego solvera dużego CUDA Vanadisa.

Najbardziej interesujący wniosek

GMRES(20) + element LU

|

v

silny, ale pamięciożerny

BiCGSTAB + element LU

|

v

mniej pamięci Kryłowa, ale LU nadal drogie

DBCG + Jacobi

|

v

słabszy preconditioner, ale ekstremalnie lekka architektura

|

v

CUDA / duże siatki

Z perspektywy dzisiejszego kodu przejście do prostego Jacobi nie wygląda jak regresja. Wygląda raczej jak wybór architektury znacznie lepiej dopasowanej do dużego EBE na GPU.

Gdybym miał sprawdzić tylko jedną nową rzecz, wybrałbym BiCGSTAB + ten sam Jacobi, matvec_0, bc_dirichlet i bez elementowego LU. To byłoby najbardziej uczciwe porównanie algorytmu z aktualnym DBCG, bez równoczesnej zmiany solvera, preconditionera i modelu pamięci.